quickly plateaus to very low values with a model that still fails to generalize. A better sampling alternative would be to perform online triplet mining over a mini-batch following the literature on metric learning. In our case, we fix each $x_{i_{\text{pri}}}$ and $x_{i_{\text{cln}}}$ in the mini-batch and search for any $x_{j_{\text{pois}}}$ with a non-zero loss (hard mining) or with a loss that is larger than zero and less than the margin (semi-hard mining). In other words, any $x_{j_{\text{pois}}}$ with any combinations of primary task, text block, and injected task would constitute a valid negative point to the anchor and positive pair of $x_{i_{\text{pri}}}$ and $x_{i_{\text{cln}}}$. We found that warming up the training with semi-hard sampling and then introducing hard sampling worked well and led to significantly better generalization. We refer to [45] for more details on triplet sampling. After mining the triplets, we optimize the model using the standard triplet loss, and repeat for each mini-batch:

$$\sum_{i,j} \left\| E(\text{Act}^{x_{i_{\text{pri}}}}) - E(\text{Act}^{x_{i_{\text{cln}}}}) \right\|_2^2 - \left\| E(\text{Act}^{x_{i_{\text{pri}}}}) - E(\text{Act}^{x_{j_{\text{pois}}}}) \right\|_2^2 + \alpha \tag{1}$$

where E is the embedding model, α is a margin hyperparameter, and i, j are indices of the mined triplet combinations in the mini-batch.

D. Hyperparameters

We set the margin α to 0.3. We used a batch size of 2500 to perform the triplet mining and a batch size of 1024 to train the model. We start the training with semi-hard triplet mining and add hard mining after 3000 training steps. We train models for 5 epochs while keeping track of the ROC AUC of the validation data to save the best model. We use Adam optimizer [55] with a learning rate of 0.0005, and we decay it by 0.05 every 800 steps.

E. Classification

At test time, the classification process is standard to metric learning verification systems. The procedure of capturing the activations is done for each instance x_i for the primary task and the full instance to get $\text{Act}^{x_{i_{\text{pri}}}}$ and Act^{x_i}. We then compute the L_2 distance between the learned embeddings, $\|E(\text{Act}^{x_{i_{\text{pri}}}}) - E(\text{Act}^{x_i})\|_2^2$, and use thresholding to classify x_i as clean or poisoned based on these distances.

F. ROC Curves of Probes

This section presents in Figure 9 the ROC curves of linear probing performance across different activation layers for five models: Phi-3 3.8B, Mistral 7B, Llama-3 8B, Mixtral 8x7B, and Llama-3 70B. The results show that the linear classifier consistently performs best when trained on activations from model layer depths 50% onwards and worse on earlier layers (especially layer 0).

We also show the ROC curve of metric learning probes in Figure 8.

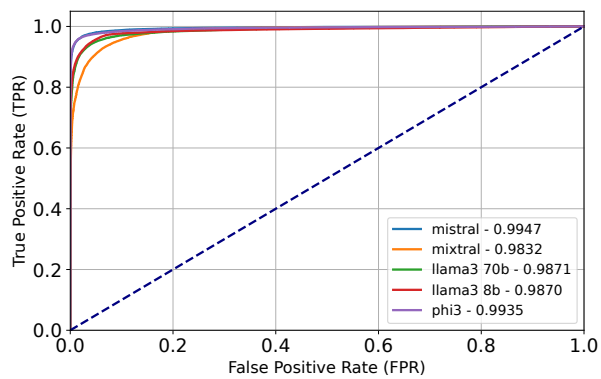
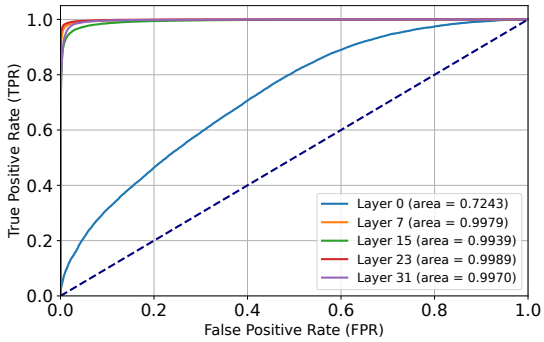
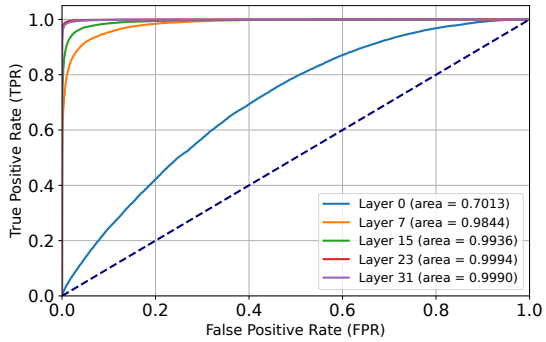


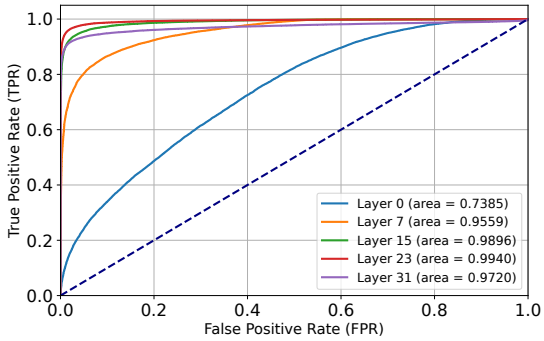
Fig. 8: ROC curves of the **metric learning probing** (the best model) across different LLMs.



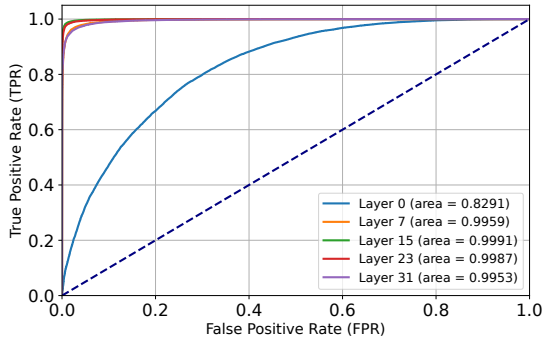
(a) Phi-3 3.8B



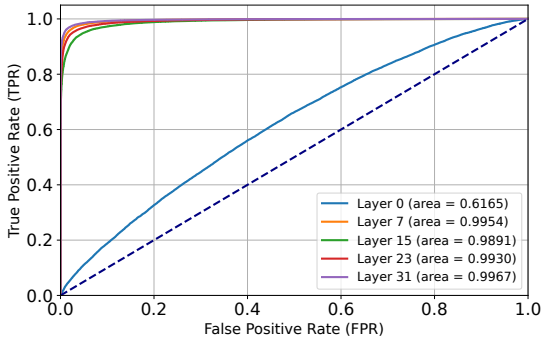
(b) Mistral 7B



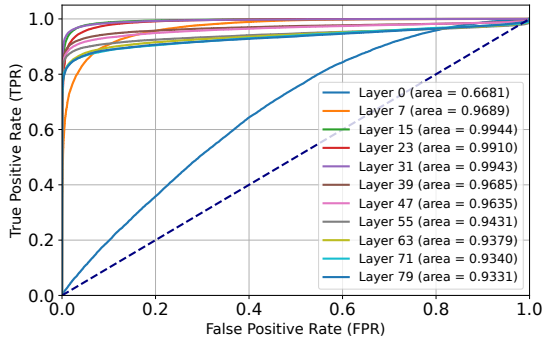
(c) Llama-3 8B



(d) Mixtral 8x7B



(e) Phi-3 14B



(f) Llama-3 70B

Fig. 9: ROC curves of the **linear probing** classifier on different layers.

G. Results of Training Probes on Activations Without a Prompt Template

Training models without the task eliciting prompt template allows us to evaluate the performance of probes in a more generalized setting. This approach helps in understanding how well models can perform on identifying task drift purely based on layers' activations, without the aid of pre-defined prompts that might facilitate the detection of task drift. The results below highlight the ROC AUC scores of the linear probe (logistic regression classifier) and the metric learning probe (the triplet embedding model) across different activation layers and layer ranges.

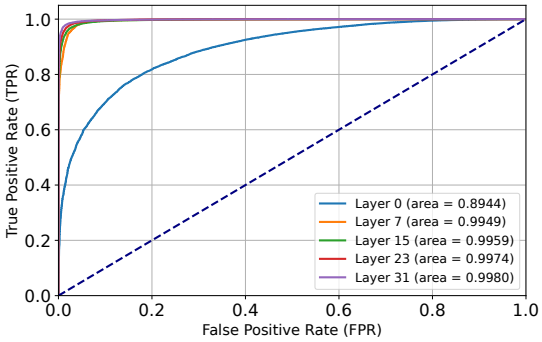
As shown in Table XVII, the linear probe still has a strong performance in this setup from most layers. However, the metric learning probe's performance significantly dropped when training with the last 15 layers compared to when using the prompt template (see Table IV). Therefore, we trained the metric learning probe on individual layers and found that some layers (7 and 15) still maintain a strong signal. We show in Figure 11 the t-SNE visualizations of the task activation deltas of Mistral 7B when collecting activations without the template, which shows relatively more segmentation and noise compared to reading with the template (see Figure 2). This may explain why the triplet network training might not have converged.

In conclusion, based on experiments on Mistral, both reading mechanisms seem to perform strongly but may require more hyperparameter search over layers in the case of the metric learning probe. It also remains to be studied which activation collection strategy results in better performance across models and better adversarial robustness.

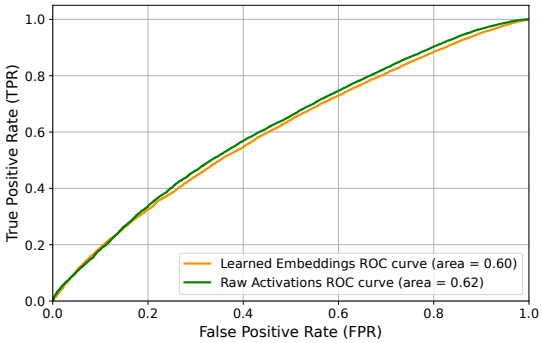
Not using a specific template has a significant computational advantage at test time for the deployed application as no extra forward passes over the context are needed to collect the activations (i.e., the primary's task's and the full instance's activations will be collected at the end of their respective context that is in any case needed to generate output from the model). Since both probes are lightweight compared to the LLM itself, the additional computational cost of classification is negligible.

Future work could also investigate tweaking the priming template (if its presence is critical) to reduce the inference cost. Instead of wrapping the template around the full context, the priming part of the prompt could come at the end (i.e., adding a suffix to the query, as in the example below), which would allow reusing the computation by caching. In all cases, we note that the cost of processing tokens is significantly less than generating ones.

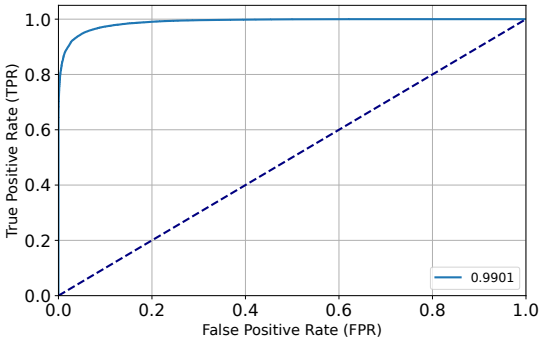
<ACTUAL QUERY>
You were given a task above, but first before you answer, please complete the following sentence by briefly writing each request(s) you received and you are going to execute next: "All requests that I am going to execute now are:"



(a) Linear probe



(b) Metric learning probe (layers 17 - 32)



(c) Metric learning probe (layer 15)

Fig. 10: ROC curves of the linear classifier probe (a) and the metric learning probe (b and c) trained on layer-wise activations, which were collected without using the task eliciting prompt template described in Section IV-A. When training the metric learning probe with the last 15 layers (b), the ROC AUC is low and almost the same as when using the raw activations. However, when training it on specific layers individually (layer 15 in c), the performance is significantly higher.

H. Distances vs. Verifier (Execution Status)

We show in Figure 12 distances for attacks that were executed, not executed, or not detected. Distances were comparable, meaning that our method can sidestep whether the attack was successful or not (e.g., due to a strong meta-prompt that instructs the model to not execute anything in the data). We also show in Table XX an example of the verifier's output.